

LAPORAN PRAKTIKUM TUGAS
MATERI 5 :
SINKRONISASI SISTEM OPERASI
(SISTEM BOOKING HOTEL)



Dosen Pengampu :
Triawan Adi Cahyanto M.Kom

Disusun Oleh :
PRADANA BAGUS SUSILO(A)

Tanggal : 13-11-2025

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

Daftar Isi

1. Pendahuluan	
1.1 Latar Belakang	
1.2 Tujuan Praktikum	
2. Soal Pemrograman: Sistem Booking Hotel	
2.1 Analisis Masalah	
2.1.1 Pemahaman tentang Soal	
2.1.2 Identifikasi Masalah Sinkronisasi	
2.1.3 Desain Solusi.....	
2.2 Implementasi	
2.2.1 Penjelasan Algoritma	
2.2.2 Flowchart / Diagram	
2.2.3 Potongan Kode Penting dengan Penjelasan.....	
2.2.4 Alasan Pemilihan Mekanisme Sinkronisasi	
2.3 Testing dan Output	
2.3.1 Screenshot Output	
2.3.2 Test Case yang Diuji	
2.3.3 Hasil yang Didapat.....	
2.4 Analisis Hasil.....	
2.4.1 Evaluasi Kinerja Program	
2.4.2 Masalah yang Ditemui dan Solusinya	
2.4.3 Performa Program	
2.4.4 Pelajaran yang Didapat	
3. Kesimpulan dan Saran	
3.1 Kesimpulan.....	
3.2 Saran	
4. Daftar Pustaka	

BAB 1

1.1 Latar Belakang

Dalam sistem operasi, beberapa proses dapat dijalankan secara bersamaan untuk meningkatkan efisiensi dan pemanfaatan sumber daya komputer. Namun, ketika proses-proses tersebut berbagi sumber daya yang sama, seperti memori atau file, dapat terjadi masalah seperti race condition, deadlock, dan inkonsistensi data. Untuk mengatasi hal tersebut, diperlukan mekanisme sinkronisasi yang mampu mengatur urutan eksekusi proses agar tidak saling bertabrakan.

Sinkronisasi proses berfungsi memastikan bahwa proses yang berjalan bersamaan tetap bekerja dengan benar dan tidak menimbulkan konflik dalam penggunaan sumber daya bersama. Melalui praktikum ini, mahasiswa dapat memahami konsep dasar sinkronisasi, mempelajari penerapan semaphore dan mutex, serta mengetahui pentingnya pengaturan akses terhadap sumber daya secara terkoordinasi dalam sistem operasi

1.2 Tujuan

- Memahami konsep dasar sinkronisasi proses, yaitu bagaimana beberapa proses dapat berjalan secara bersamaan tanpa saling mengganggu atau menimbulkan kondisi race (race condition).
- Mempelajari cara kerja mekanisme sinkronisasi seperti semaphore, mutex, dan monitor dalam mengatur akses ke sumber daya bersama (shared resources).
- Menganalisis permasalahan deadlock dan race condition, serta memahami cara mencegah atau mengatasinya melalui penerapan teknik sinkronisasi yang tepat.
- Mengimplementasikan program sederhana yang menerapkan konsep sinkronisasi proses menggunakan bahasa pemrograman C atau bahasa lain yang relevan.
- Menumbuhkan kemampuan berpikir logis dan sistematis dalam merancang solusi untuk permasalahan yang melibatkan beberapa proses yang berjalan secara paralel.

BAB 2

2.1 Analisis Masalah

Deskripsi Masalah:

Anda diminta membuat simulasi sistem booking hotel dengan kondisi:

- a. Hotel memiliki 5 kamar yang bisa di booking
- b. Ada 10 customer yang mencoba booking secara bersamaan (10 threads)
- c. Setiap customer akan:
 1. Mencoba booking kamar (butuh waktu random 1-3 detik)
 2. Jika berhasil, "menginap" selama 5 detik
 3. Check-out dan melepaskan kamar
 4. Jika gagal (kamar penuh), customer menunggu 2 detik lalu coba lagi
- d. Customer maksimal mencoba 3 kali, jika gagal terus maka menyerah

Requirements:

1. Gunakan Semaphore untuk mengatur jumlah kamar tersedia
2. Gunakan Mutex untuk melindungi operasi print (agar output tidak berantakan)
3. Tampilkan informasi:
 - Customer ID berapa yang berhasil booking
 - Kamar mana yang ditempati
 - Berapa kamar yang masih tersedia
 - Customer yang gagal booking
4. Output harus rapi dan mudah dibaca (gunakan emoji atau format yang jelas)

Pemahaman tentang soal

Soal tentang sistem booking hotel ini bertujuan untuk mensimulasikan proses pemesanan kamar hotel dengan menggunakan konsep *multithreading* dan *sinkronisasi* dalam sistem operasi. Dalam simulasi ini, hotel digambarkan memiliki sejumlah kamar tertentu, misalnya lima kamar, sementara terdapat sepuluh pelanggan atau *customer* yang mencoba melakukan pemesanan secara bersamaan. Setiap pelanggan direpresentasikan sebagai satu *thread* yang berjalan secara paralel, sehingga bisa saja beberapa pelanggan mencoba mengakses data kamar pada waktu yang sama. Kondisi inilah yang menimbulkan potensi konflik data atau *race condition*, sehingga dibutuhkan mekanisme sinkronisasi untuk mengatur akses terhadap sumber daya bersama.

Desain Solusi

Desain solusi dari program sistem booking hotel ini berfokus pada bagaimana mengatur proses pemesanan kamar oleh banyak pelanggan secara bersamaan tanpa menimbulkan konflik data. Dalam hal ini, perancangan solusi dibangun dengan menggunakan konsep dasar sinkronisasi antar *thread* menggunakan *semaphore* dan *mutex*. Program diawali dengan menentukan jumlah kamar dan jumlah pelanggan. Misalnya, hotel memiliki lima kamar dan sepuluh pelanggan yang ingin melakukan pemesanan. Setiap pelanggan direpresentasikan sebagai satu *thread* yang berjalan secara paralel. Tujuannya adalah untuk mensimulasikan kondisi nyata di mana beberapa orang mencoba memesan kamar hotel dalam waktu yang hampir bersamaan.

2.2 Penjelasan Algoritma

1. Inisialisasi jumlah kamar, *semaphore*, dan *mutex*. Semua kamar diatur kosong.
2. Buat thread sebanyak jumlah pelanggan (misalnya 10 thread).
3. Setiap *thread* pelanggan akan:
 - Menunggu waktu acak 1–3 detik sebelum mencoba booking.
 - Mencoba memesan kamar menggunakan *sem_trywait*.
4. Jika berhasil booking:
 - *Mutex* dikunci, kamar ditandai terisi.
 - *Thread* tidur 5 detik (simulasi menginap).
 - Setelah itu, kamar dikosongkan kembali dan *sem_post* dijalankan.
5. Jika gagal booking (karena kamar penuh):
 - Tunggu 2 detik lalu coba lagi.
 - Maksimal 3 kali percobaan, jika tetap gagal → menyerah.
6. Setelah semua pelanggan selesai, *thread* digabung (*pthread_join*) dan program berakhir.

Alasan Pemilihan Mekanisme Sinkronisasi

Alasan pemilihan mekanisme sinkronisasi pada program sistem booking hotel ini adalah untuk memastikan bahwa proses pemesanan kamar oleh banyak pelanggan (*thread*) dapat berjalan secara aman dan teratur tanpa menimbulkan konflik data. Dalam simulasi ini, beberapa *thread* berjalan bersamaan dan berpotensi mengakses variabel atau sumber daya yang sama, yaitu data kamar hotel. Tanpa mekanisme sinkronisasi, dua atau lebih *thread* dapat memesan kamar yang sama pada waktu bersamaan, sehingga menimbulkan kesalahan atau hasil yang tidak konsisten.

Penggunaan **semaphore** dipilih karena mekanisme ini sangat efektif dalam mengatur jumlah sumber daya yang terbatas, seperti kamar hotel. Dengan nilai awal yang sesuai dengan jumlah kamar, *semaphore* dapat secara otomatis menolak *thread* yang ingin memesan kamar ketika semua kamar sudah terisi. Hal ini membuat pengelolaan ketersediaan kamar menjadi efisien tanpa perlu menggunakan banyak kondisi logika tambahan.

Source Kode

[illegible][illegible]

```
// create customers
for (int i = 0; i < NUM_CUSTOMERS; ++i) {
    int *idptr = malloc(sizeof(int));
    *idptr = i + 1; // customer IDs start from 1
    if (pthread_create(&threads[i], NULL, customer_thread, idptr) != 0) {
        perror("pthread_create");
        exit(EXIT_FAILURE);
    }
    // optional small stagger to make output nicer
    usleep(100000); // 0.1s
}

// wait for all threads
for (int i = 0; i < NUM_CUSTOMERS; ++i) {
    pthread_join(threads[i], NULL);
}

// cleanup
pthread_mutex_destroy(&rooms_mutex);
sem_destroy(&rooms_sem);

printf("\nSemua customer selesai.\n");
return 0;
}
```

Output

```
pradana@Bagus:~$ nano hotel_booking.c
pradana@Bagus:~$ gcc -pthread -o hotel_booking hotel_booking.c
hotel_booking.c: in function 'main':
hotel_booking.c:145:9: warning: implicit declaration of function 'usleep'; did you mean 'sleep'? [-Wimplicit-function-declaration]
   145 |         usleep(100000); // 0.1s
       |         ^~~~~~
       |         sleep
pradana@Bagus:~$ ./hotel_booking
=== SYSTEM BOOKING HOTEL ===
Total kamar: 5
Total customer (threads): 10

[11:58:48] Customer-2 mencoba booking...
hotel_booking.c:145:9: warning: implicit declaration of function 'usleep'; did you mean 'sleep'? [-Wimplicit-function-declaration]
   145 |         usleep(100000); // 0.1s
       |         ^~~~~~
       |         sleep
```

Test Case Yang Diuji

- 10 customer mencoba booking secara bersamaan.
- Ada customer yang gagal booking karena kamar penuh.
- Ada customer yang berhasil setelah mencoba beberapa kali.
- Semua kamar kembali tersedia setelah seluruh customer check-out.

Hasil yang Didapat

- Versi Deadlock:
Program berhenti (macet) karena dua thread saling menunggu kunci — terjadi *deadlock*.
- Versi Mutex:
Semua thread berjalan bergantian, tidak ada *deadlock*. Proses booking dan checkout berhasil sesuai kapasitas kamar.
- Versi Semaphore:
Thread berjalan lancar, maksimal hanya sejumlah kamar yang bisa dipakai bersamaan. Tidak ada *deadlock*, sistem lebih efisien.

2.4 Analisis Hasil

Apakah Program Berjalan sesuai ekspektasi?

Ya, program berjalan sesuai dengan ekspektasi setelah dilakukan perbaikan. Semua *customer thread* dapat melakukan proses booking kamar, menginap, dan *check-out* dengan benar tanpa terjadi *deadlock* atau error. Selain itu, jumlah kamar yang tersedia selalu terkontrol sesuai kapasitas hotel berkat penggunaan *semaphore* dan *mutex* untuk sinkronisasi antar-thread.

Masalah apa yang ditemui dan solusinya

Masalah yang ditemui pada awalnya adalah program mengalami *deadlock* dan beberapa *thread* berhenti tanpa menyelesaikan proses booking. Hal ini terjadi karena tidak ada pengaturan sinkronisasi yang baik antara *thread*, sehingga beberapa *customer* mencoba mengakses data kamar secara bersamaan.

Solusinya adalah dengan menambahkan **Semaphore** untuk membatasi jumlah kamar yang bisa diakses secara bersamaan dan **Mutex** untuk melindungi bagian *critical section* seperti proses print dan update jumlah kamar. Setelah solusi ini diterapkan, program dapat berjalan lancar tanpa *deadlock* dan seluruh proses booking berjalan sesuai alur.

Performa Program

Performa program tergolong baik dan stabil. Semua *thread* dapat berjalan secara paralel tanpa saling mengganggu karena sinkronisasi yang tepat antara *semaphore* dan *mutex*. Waktu eksekusi juga efisien, karena setiap *customer* hanya menunggu ketika semua kamar penuh, lalu mencoba lagi sesuai batas percobaan. Selain itu, penggunaan *sleep* yang bervariasi membantu mensimulasikan kondisi nyata tanpa membebani CPU secara berlebihan.

Pelajaran Yang didapat

Pelajaran yang didapat dari percobaan ini adalah pentingnya penerapan konsep sinkronisasi dalam pemrograman multithreading. Tanpa sinkronisasi yang tepat, program dapat mengalami *deadlock* atau *race condition* yang menyebabkan hasil tidak sesuai. Dengan menggunakan mutex dan semaphore, akses ke sumber daya bersama dapat diatur dengan aman, sehingga program berjalan lebih stabil, efisien, dan bebas dari konflik antar-thread.

Kesimpulan

dari hasil percobaan yang dilakukan dapat disimpulkan bahwa penerapan konsep sinkronisasi dalam pemrograman multithreading sangat penting untuk menjaga stabilitas dan efisiensi sistem. Pada versi awal program, terjadi *deadlock* karena beberapa *thread* berusaha mengakses sumber daya bersama (data kamar hotel) secara bersamaan tanpa pengaturan yang benar. Hal ini menyebabkan seluruh proses berhenti dan tidak dapat diselesaikan.

Setelah dilakukan perbaikan dengan menerapkan *mutex* dan *semaphore*, masalah tersebut berhasil diatasi. *Mutex* berfungsi untuk melindungi bagian kritis (*critical section*) agar hanya satu *thread* yang dapat mengaksesnya dalam satu waktu, sedangkan *semaphore* digunakan untuk membatasi jumlah *thread* yang dapat mengakses sumber daya sesuai kapasitas kamar yang tersedia.

Dengan penerapan mekanisme sinkronisasi ini, program berjalan sesuai ekspektasi, semua *thread* dapat melakukan proses booking dan *check-out* secara bergantian tanpa konflik. Selain itu, performa program menjadi lebih efisien dan realistis dalam mensimulasikan sistem reservasi kamar hotel. Secara keseluruhan, percobaan ini memberikan pemahaman mendalam tentang pentingnya pengendalian akses bersama dalam sistem multithreading agar terhindar dari *deadlock* dan *race condition*.